

Learning scheduling algorithms for data processing clusters

Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja
Venkatakrishnan, Zili Meng, Mohammad Alizadeh

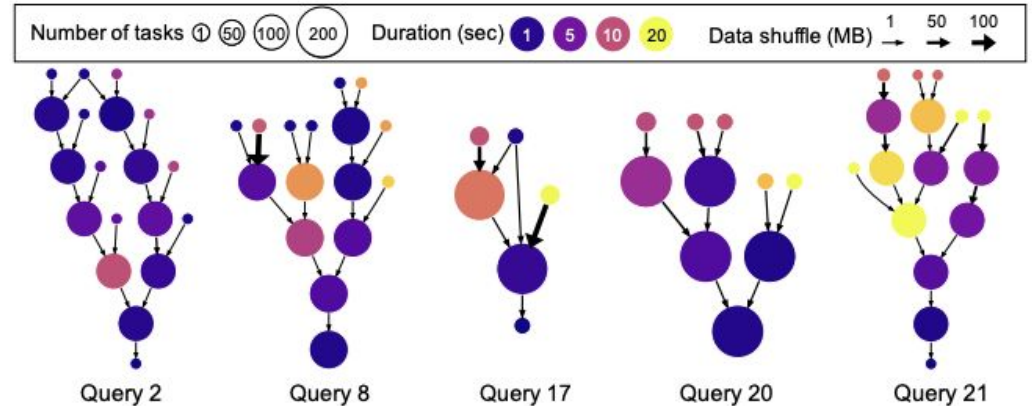
ACM SIGCOMM '19

Decima: Reinforcement Learning for DAG-Based Cluster Scheduling

- Paper proposes a reinforcement learning-based scheduler for DAG-structured jobs in large-scale data processing clusters
- Optimizes average job completion time rather than relying on heuristic-based priority structure
- System is implemented within Spark and evaluated under realistic workload conditions

Background Information/Terminology

- Job
- Cluster
- Apache Spark
- Scheduler
- Executor
- Task
- Stage
- DAG



Problem Formulation and Objective

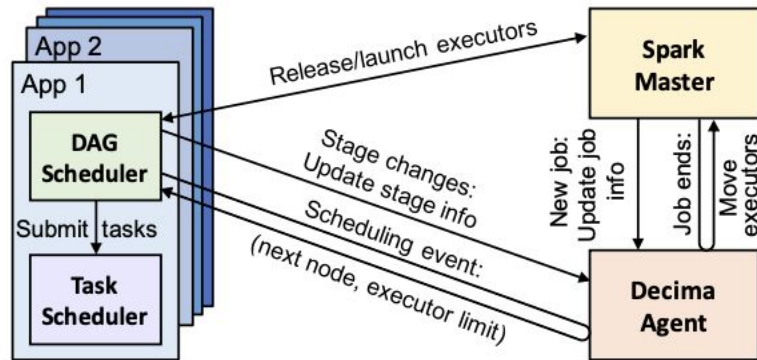
- The cluster executes multiple jobs simultaneously, and each job is represented as a DAG of stages
- Executors are limited shared resources that must be allocated across jobs
- The objective is to minimize the average job completion time over all jobs
- Equivalent to minimizing the accumulated number of unfinished jobs over time

Motivation

- Cluster utilization is expensive to get wrong, as even small improvements in utilization can save substantial cost at enterprise scale
- Current schedulers rely on hand-designed heuristics because they are general and easy to implement
- Heuristics leave performance on the table for a specific workload because they do not directly optimize the target objective
- Ignore useful structure that is often available like job dependencies, effects of parallelism choices for different input sizes
- Workload-specific policies are hard to build manually

Spark Scheduling

- Job is split into stages, and each stage consists of many parallel tasks
- Workers run executor JVM processes, and each executor may execute multiple tasks concurrently
- Shuffle boundaries introduce network transfer and disk merge overhead that affect runtime
- Scheduler operates at the stage level, leave individual task scheduling for Spark's default logic



Applying DAG Structure

- Stage can only execute after all of its parent stages have completed
- The length of the critical path in the DAG is inherently a bare minimum requirement for job completion time
- Scheduling decisions that delay critical path stages directly increase job completion time
- Parallel edges of a DAG compete for the same shared executors

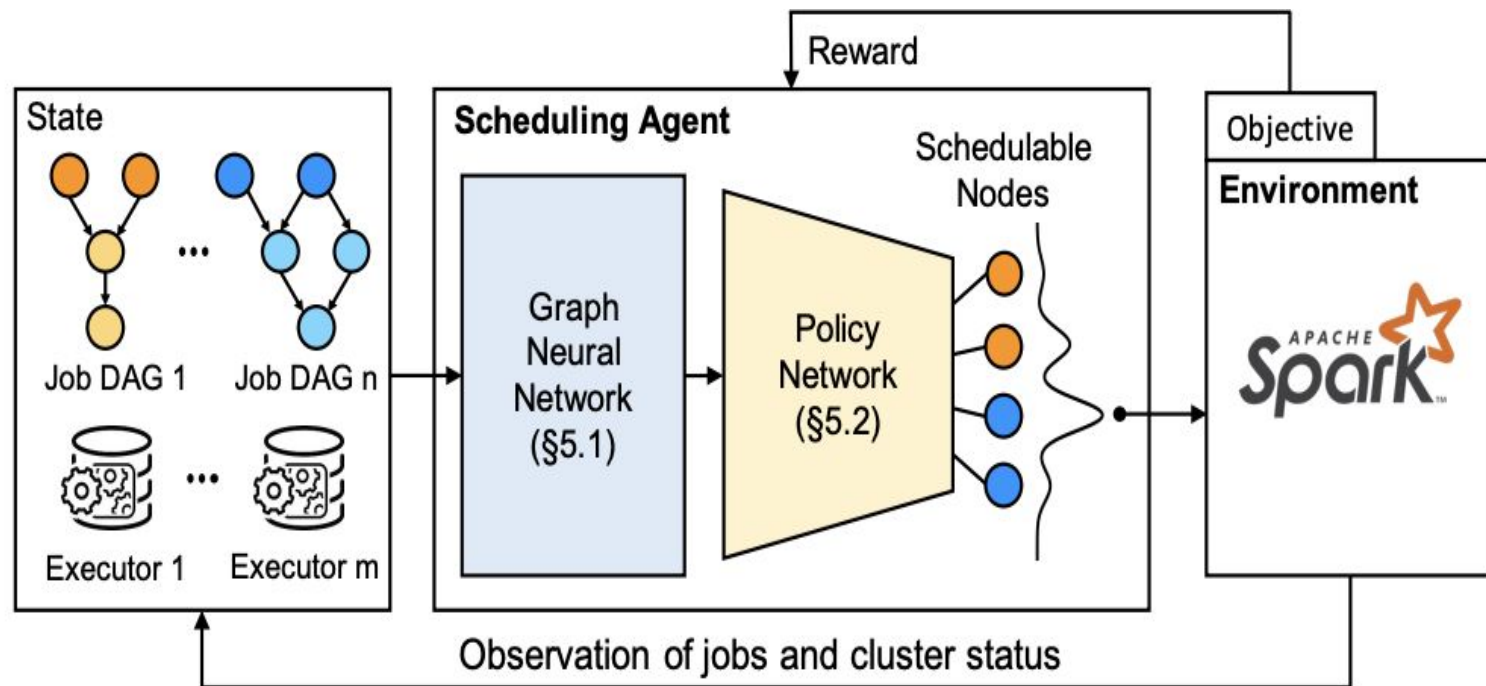
Limitations of Standard Heuristics

Baseline Heuristic Schedulers include:

- Spark's default FIFO scheduling
- Shortest-job-first critical-path heuristic (SJF-CP)
- Simple fair scheduling
- Naive weighted fair scheduling
- Carefully tuned weighted fair scheduling (optimized α)
- Tetris (standard multi-resource packing algorithm)
- Graphene* (adaptation of Graphene for discrete executor classes)

None of these heuristics explicitly optimize long-term average job completion time

Decima Structure

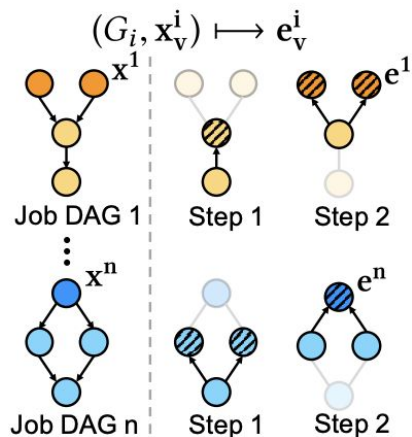


GNN Representation

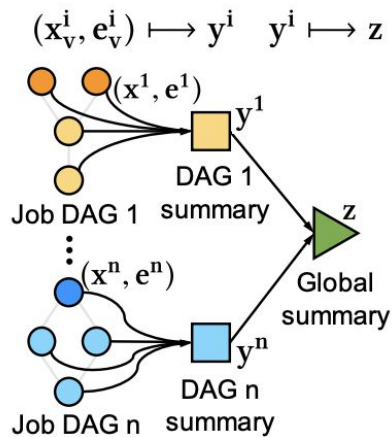
- From inputted DAGs, per-node feature vector is constructed manually using information such as number of remaining tasks, average task duration, number of currently working executors, number of available executors, and locality of these executors

entity	symbol	entity	symbol
job	i	per-node feature vector	$\mathbf{x}_v^i$
stage (DAG node)	v	per-node embedding	$\mathbf{e}_v^i$
node v 's children	$\xi(v)$	per-job embedding	$\mathbf{y}^i$
job i 's DAG	G_i	global embedding	$\mathbf{z}$
job i 's parallelism	l_i	node score	q_v^i
non-linear functions	f, g, q, w	parallelism score	w_l^i

GNN Representation (cont.)



(a) Per-node embedding.



(b) Per-job and global embeddings.

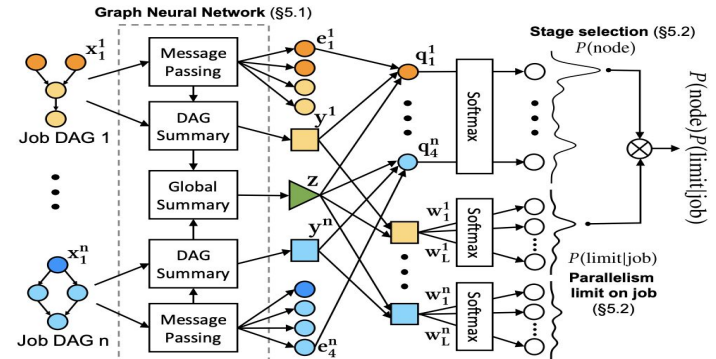
$$\mathbf{e}_v^i = g \left[\sum_{u \in \xi(v)} f(\mathbf{e}_u^i) \right] + \mathbf{x}_v^i,$$

Decision Process

- Decima represents the scheduler as an agent that uses a neural network to make decisions (policy network)
- Decisions happen on scheduling events (stage runs out of tasks, a stage completion or a job arrival), and the agent takes input and outputs a scheduling action
- Training setup: Decima trains the neural network using RL through a large number of offline (simulated) experiments.
- State: Status of the DAGs in the scheduler's queue and the executors
- Action: choose (i) which runnable stage to schedule next and (ii) an executor-limit (parallelism cap) for that stage's job.

Decision Process (cont.)

- At each scheduling event, Decima converts the current scheduler state (job DAGs, runnable stages, executor status, stage/task statistics) into per-stage feature vectors, then uses a GNN to produce stage/job/global embeddings used by the policy.
- Policy network scores runnable stages and executor-limit choices from these scores, Decima selects the next stage and its job's executor-limit (parallelism limit)



Decision Process (cont.)

- At each scheduling event, Decima may make a sequence of actions to assign a group of free executors, repeating until: all free executors are assigned, or no runnable stages remain.
- After each action, Decima gives reward r_k based on the high-level objective; for average JCT, it penalizes elapsed time multiplied by number of jobs in the system
- Minimizes the average number of jobs in the system and, by Little's law, effectively minimizes average job completion time

Policy Gradient Optimization

Decima uses the REINFORCE policy gradient algorithm to optimize expected return

Algorithm 1 Policy gradient method used to train Decima.

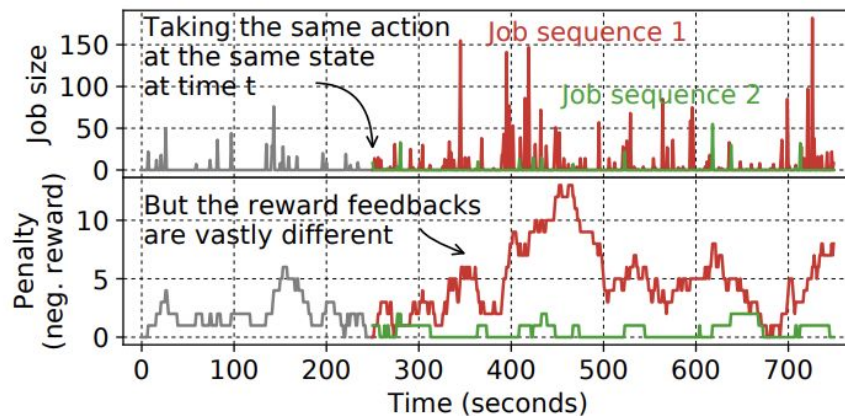
```
1: for each iteration do
2:    $\Delta\theta \leftarrow 0$ 
3:   Sample episode length  $\tau \sim \text{exponential}(\tau_{\text{mean}})$ 
4:   Sample a job arrival sequence
5:   Run episodes  $i = 1, \dots, N$ :
      $\{s_1^i, a_1^i, r_1^i, \dots, s_\tau^i, a_\tau^i, r_\tau^i\} \sim \pi_\theta$ 
6:   Compute total reward:  $R_k^i = \sum_{k'=k}^\tau r_{k'}^i$ 
7:   for  $k = 1$  to  $\tau$  do
8:     compute baseline:  $b_k = \frac{1}{N} \sum_{i=1}^N R_k^i$ 
9:     for  $i = 1$  to  $N$  do
10:       $\Delta\theta \leftarrow \Delta\theta + \nabla_{\theta} \log \pi_{\theta}(s_k^i, a_k^i)(R_k^i - b_k)$ 
11:    end for
12:  end for
13:   $\tau_{\text{mean}} \leftarrow \tau_{\text{mean}} + \epsilon$ 
14:   $\theta \leftarrow \theta + \alpha \Delta\theta$ 
15: end for
```

Curriculum Learning for Training Stability

- Early random policies cannot keep up with arrivals and cause backlog explosion
- Training begins with short episode horizons
- Episode length is gradually increased as the policy improves, which prevents catastrophic instability during early learning

Variance Reduction

- Job arrivals introduce external randomness that inflates gradient variance
Decima fixes job arrival sequences across training episodes
- Stabilize training and convergence by separating baseline and computing it for each arrival trace

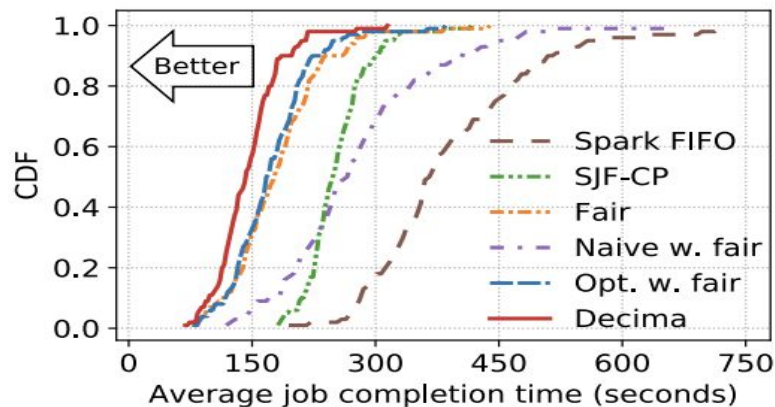


Evaluation (Setup)

- Evaluation uses a 25-worker Spark cluster with two executors per worker
- Each worker has eight CPU cores and 16 GB of memory
- Experiments evaluate both batched arrivals and continuous stochastic arrivals
- Baselines include: FIFO, fair scheduling, weighted fair scheduling, Tetris etc.
- Benchmark: TPC-H Spark queries

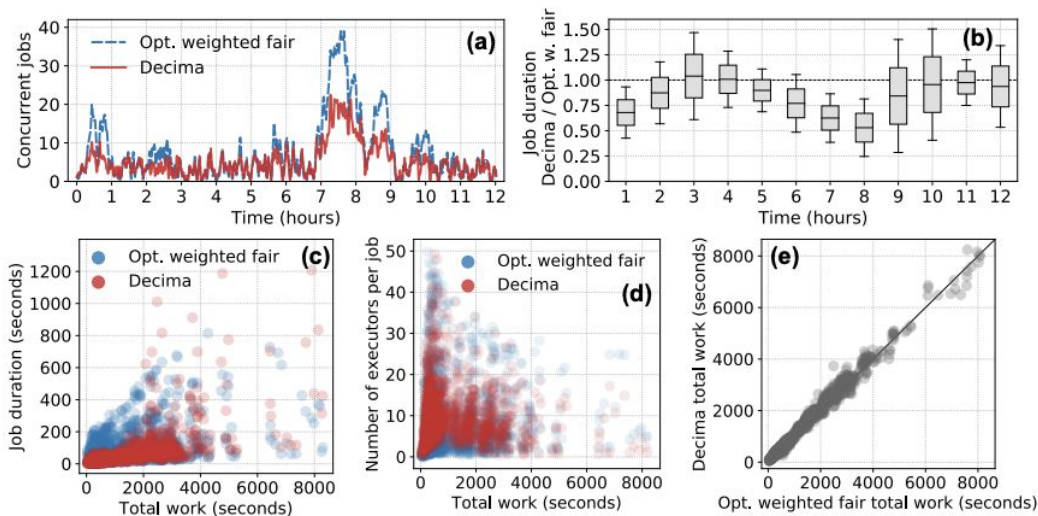
Batched Arrivals Evaluation

- Decima consistently reduces average job completion time compared to all baselines
- The improvements are significant across different workload mixes
- Structure-aware scheduling accelerates critical path completion
- JCT lower by 21% than optimal weighted fair scheduling



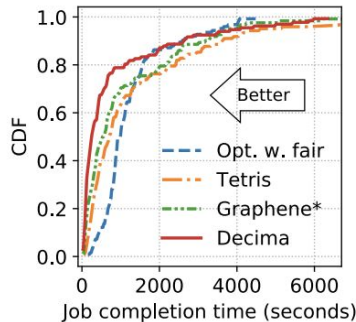
Continuous Arrivals Evaluation

- Decima remains robust under stochastic arrival processes
- It maintains lower queue length under heavy load conditions
- Variance reduction and long-horizon optimization contribute to robustness
- Achieves JCT 29% lower than optimal weighted fair scheduling

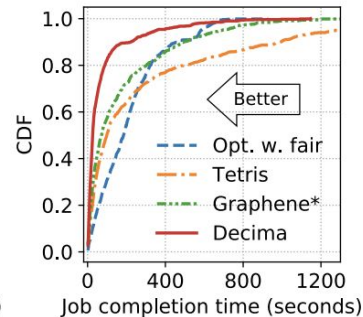


Multi-dimensional Resource Packing

- Can also work to handle for additional resource requirements
- Applied modified Decima with each task requiring some minimum CPU and memory
- Have executor classes of executors with varying memory
- Achieves a 32% lower average JCT than Graphene*



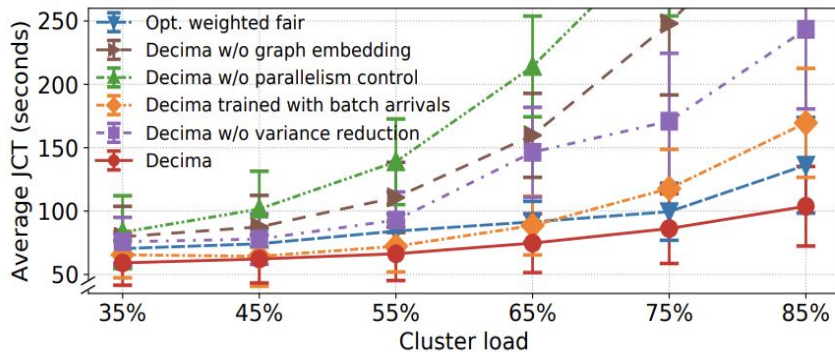
(a) Industrial trace replay.



(b) TPC-H workload.

Ablation Study

- Trained 5 different variants of Decima on simulated cluster with 1000 continuous TPC-H job arrivals (removing parallelism control, graph embeddings, variance reduction, and training on matched arrival structure)
- Removing any component gives worse average JCTs than the tuned weighted-fair heuristic at a high cluster load



Critiques

- Evaluation relies on simulator fidelity and accurate profiling
- May not be able to handle cases of extreme diversity
- Limited interpretability relative to classical heuristics
- Optimization of JCT can hurt other factors such as tail latency or fairness in particular situations

Future Directions

- Can consider broader objectives besides just minimizing JCT and makespan
- Work on more elaborate cluster data (consider complex multi-resource constraints, vary hardware, and restrict placements)
- Generalize over different sizes of cluster or types of workloads
- Can add preemptiveness to similar system

Implications

- Learned schedulers can outperform classical schedulers in complex cluster configurations
- Representation learning is a crucial aspect of scheduling and perhaps system control in general
- Must configure environment, overhead assumptions, and objective for Decima to adapt

Conclusion

Decima formulates cluster scheduling as an RL problem and uses a policy that is fed GNN embedding outputs as state representations and parallelism limits to make scheduling decisions based on DAG structure of jobs.

Makes significant improvements in JCT over continuous and batched arrival settings.